



ĐỒ ÁN

THIẾT KẾ HỆ THỐNG NETWORK ON CHIP TÍCH HỢP MULTIPLY CORE

GVHD: ThS.Tạ Trí Đức

- Hà Y Như Ý 23521843



Nội dung báo cáo

1 Tổng quan đề tài

2 Giải pháp đề xuất

3 Kết quả giải pháp đề xuất

4 Kết luận và hướng phát triển

01

TỔNG QUAN ĐỀ TÀI NGHIÊN CỨU





1. Tổng quan đề tài nghiên cứu

- Giới thiệu tổng quan về đề tài
 - **Đề tài:** Thiết kế và hiện thực hóa mạng NoC (Network-on-Chip) kiến trúc 3x3 Mesh cho hệ thống đa lõi.
 - **Công nghệ:** Sử dụng giao thức OBI để truyền thông, tích hợp bộ vi xử lý RISC-V và bộ nhớ chia sẻ (Shared RAM).
 - **Mục tiêu:** Xây dựng mạng lưới truyền tin tốc độ cao, thay thế bus truyền thống nhằm tăng hiệu năng hệ thống.



MỤC TIÊU VÀ GIỚI HẠN

- **Thiết kế:** Hoàn thiện kiến trúc mạng NoC 3x3 Mesh, tích hợp 02 nhân xử lý RISC-V và bộ nhớ chia sẻ.
- **Tối ưu:** Đạt mục tiêu về hiệu năng (PPA): $F_{max} > 50\text{MHz}$ và tối ưu tài nguyên logic (> 30000 LEs).
- **Kiểm chứng:** Xây dựng hệ thống Testbench tự động để đo lường các chỉ số quan trọng: và cơ chế xử lý kẹt mạng (Congestion management), độ ưu tiên.

02

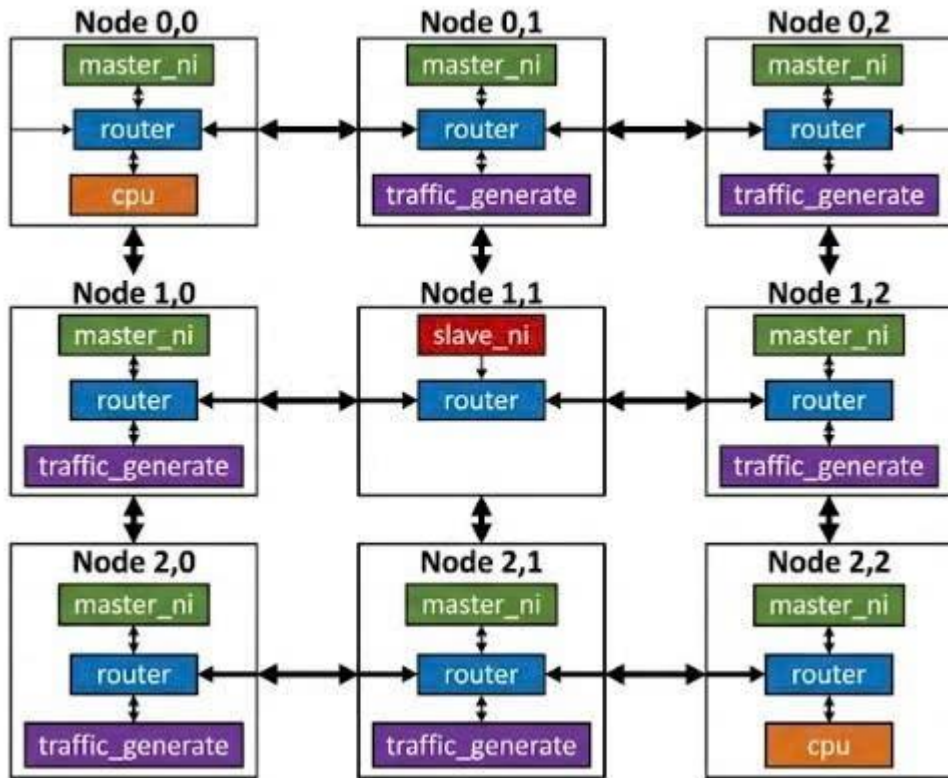
GIẢI PHÁP ĐỀ XUẤT CỦA ĐỀ TÀI





2. Giải pháp đề xuất của đề tài

Tổng quan kiến trúc NoC:



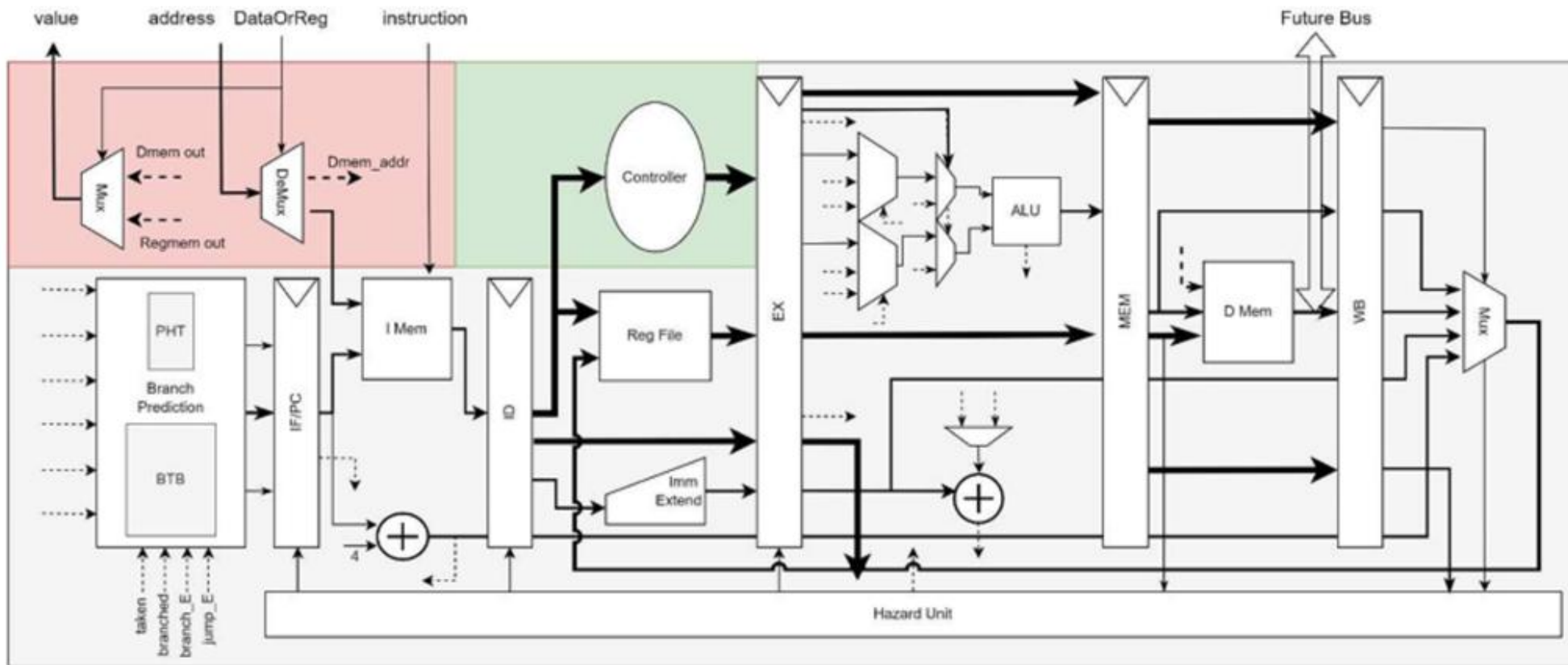
Kiến trúc NoC này sử dụng **cấu trúc liên kết (Topology) dạng Mesh 3x3** bao gồm **9 node**, với chức năng cụ thể như sau:

- **Node 00 và Node 22:** Được tích hợp CPU, đảm nhiệm vai trò tính toán chính trong hệ thống.
- **Node 11:** Đóng vai trò lưu trữ bộ nhớ dùng chung (Shared RAM).
- **Các node còn lại:** Được kết nối với bộ tạo lưu lượng (Traffic Generator) để phát sinh các gói tin giả lập, phục vụ cho việc kiểm tra và đánh giá tình trạng nghẽn mạng.



2. Giải pháp đề xuất của đề tài

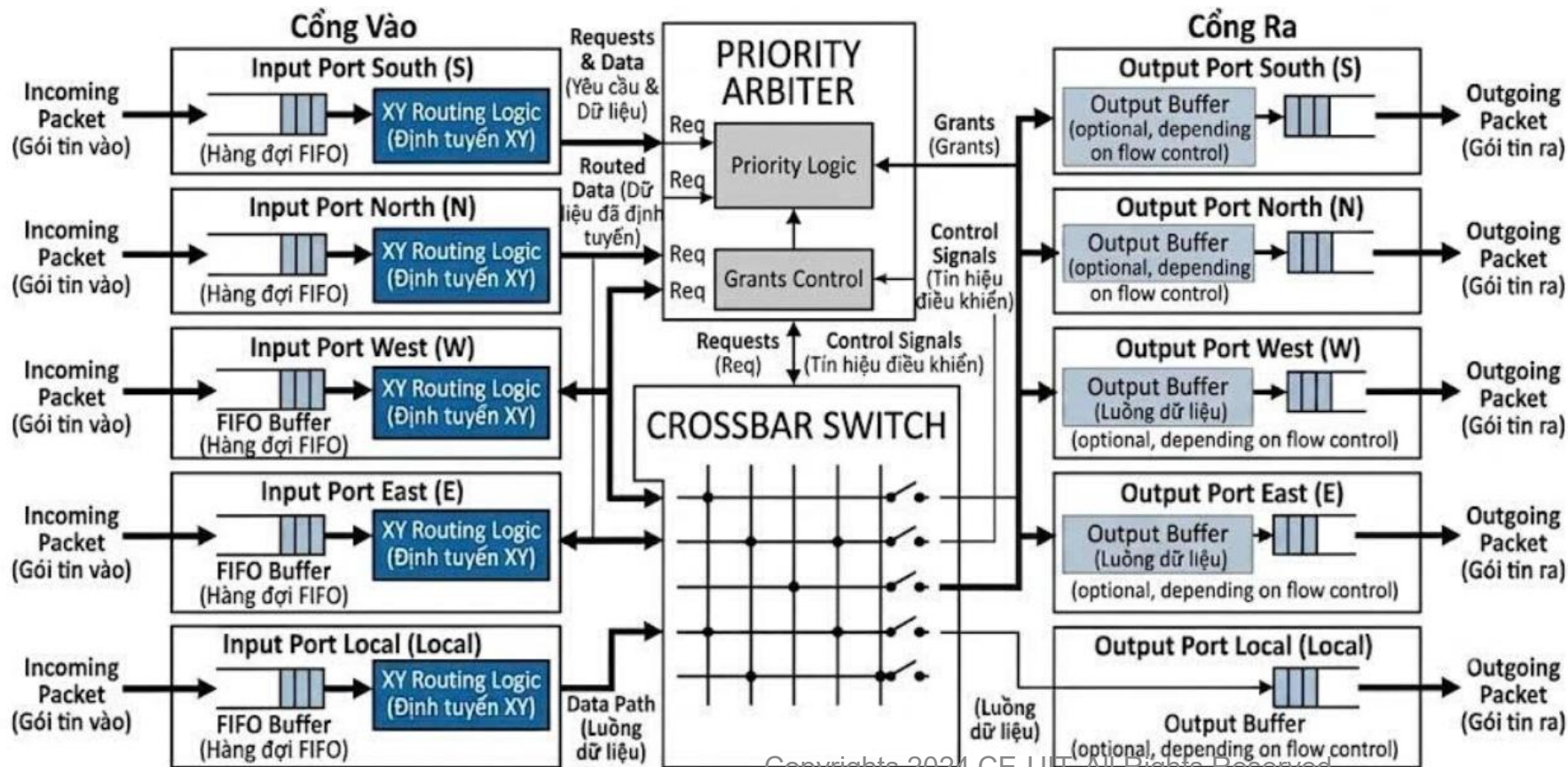
Kiến trúc risc-V32 Bit pipeline 5 tầng





2. Giải pháp đề xuất của đề tài

Sơ đồ của router





2. Giải pháp đề xuất của đề tài

- **Kiến trúc lõi Router bao gồm 4 khối chức năng chính:**
 - **input_port (Cổng vào):** Tiếp nhận dữ liệu; tích hợp bộ đệm fifo_buffer và giải mã định tuyến xy_routing.
 - **arbiter (Bộ trọng tài):** Phân xử và cấp quyền ưu tiên luồng tin theo thuật toán priority_round_robin.
 - **crossbar_switch (Ma trận chuyển mạch):** Đóng cắt mạch và điều hướng chuẩn xác dữ liệu từ ngõ vào ra ngõ ra.
 - **output_port (Cổng ra):** Xuất dữ liệu và giao tiếp kiểm soát luồng với Router kế tiếp.



2. Giải pháp đề xuất của đề tài

Các thuật toán sử dụng trong đề tài:

Thuật toán định tuyến xy:

- **Cơ chế:** Gói tin di chuyển triệt để theo trục ngang (X) trước, sau đó mới rẽ sang trục dọc (Y) để tới đích.
- **Ưu điểm:** Mạch logic siêu nhẹ và đảm bảo 100% không bao giờ xảy ra bế tắc mạng (Deadlock-free).

Chuyển mạch lỗ sâu (Wormhole Switching):

- **Cơ chế:** Gói tin được băm nhỏ thành các "Flit" head, body, tail nối đuôi nhau; Flit đầu dò đường, các Flit sau luồn lách đi theo.
- **Ưu điểm:** Độ trễ truyền tải siêu thấp và cho phép sử dụng bộ đệm (FIFO) cực nhỏ để tiết kiệm tài nguyên.

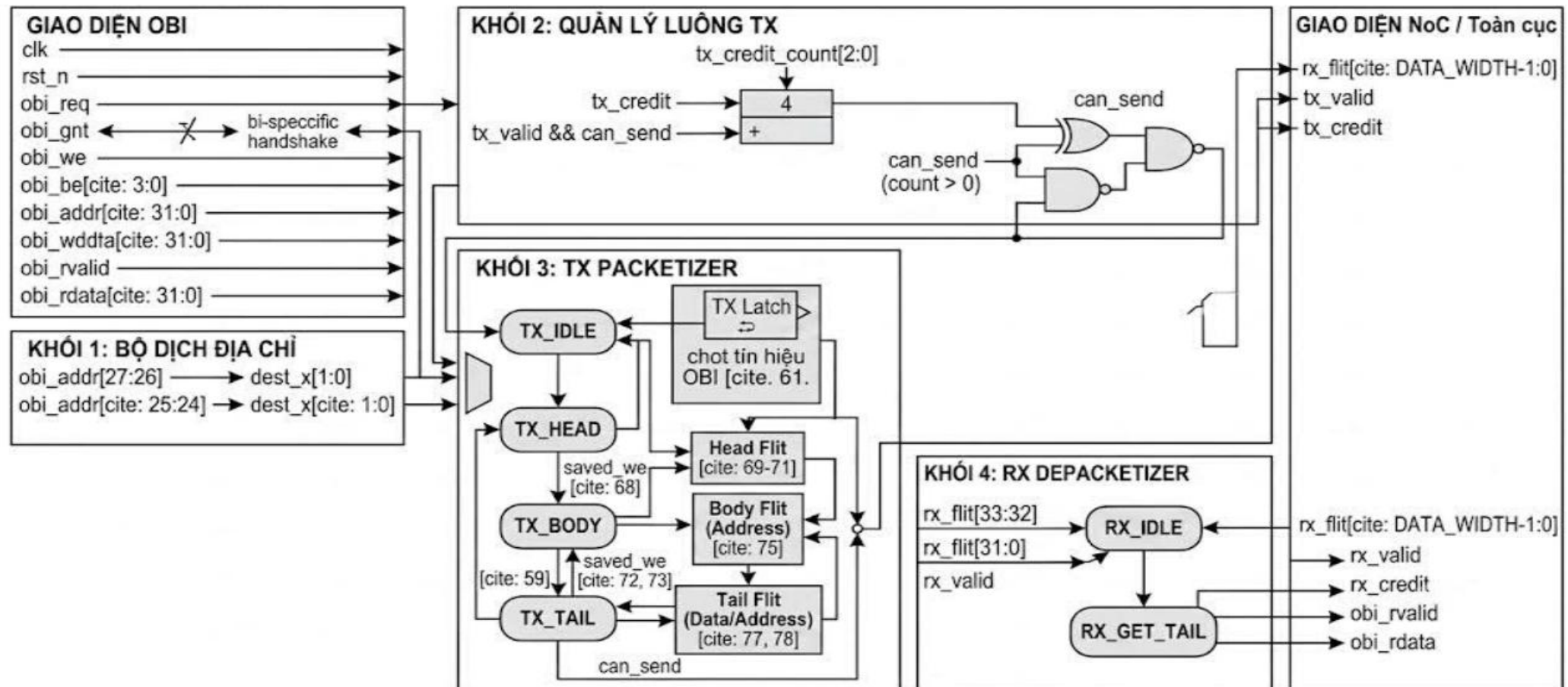
Kiểm soát luồng tín nhiệm (Credit-based Flow Control):

- **Cơ chế:** Dựa vào "điểm tín nhiệm" (Credit) báo về từ số ô trống của bộ đệm đích để quyết định gửi dữ liệu.
- **Ưu điểm:** Truyền dữ liệu tốc độ cao (không cần bắt tay chờ đợi) và đảm bảo 100% không bao giờ rớt gói tin (Drop-free).



2. Giải pháp đề xuất của đề tài

- NETWORK INTERFACE: Master_{ni}:





2. Giải pháp đề xuất của đề tài

NGUYÊN LÝ HOẠT ĐỘNG MASTER NI (CẦU NỐI CPU - NoC)

1. Dịch địa chỉ (Address Decoder):

- Ánh xạ bộ nhớ linh hoạt bằng cách trích xuất trực tiếp tọa độ đích (Dest X, Dest Y) từ các bit cao của địa chỉ do CPU phát ra.

2. Đóng gói gói tin (TX Packetizer - Hướng gửi đi):

- Biến đổi lệnh OBI của CPU thành chuỗi Flit (34-bit) bằng Máy trạng thái (FSM):
 - **Head Flit:** Mang thông tin định tuyến (Tọa độ nguồn/đích, Cờ Đọc/Ghi, Byte Enable).
 - **Body Flit:** Mang địa chỉ bộ nhớ (Chỉ dùng cho lệnh Ghi).
 - **Tail Flit:** Mang Dữ liệu (lệnh Ghi) hoặc Địa chỉ (lệnh Đọc), chốt hạ giao dịch.

3. Giải nén gói tin (RX Depacketizer - Hướng nhận về):

- Xử lý phản hồi từ mạng cho lệnh Đọc: Nhận diện Head Flit, sau đó bóc tách dữ liệu 32-bit từ Tail Flit để trả tín hiệu rdata và bật cờ rvalid cho CPU.

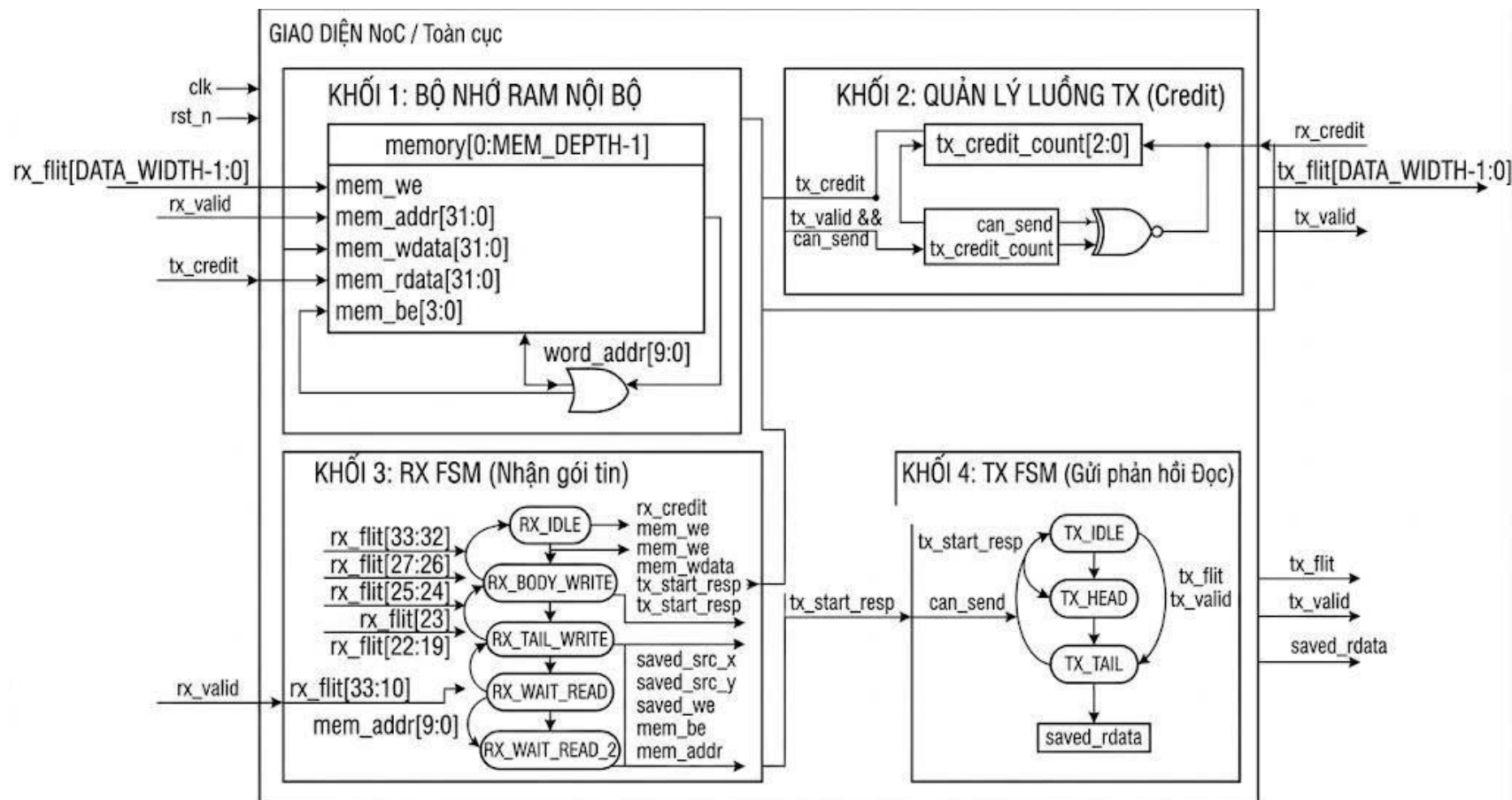
4. Quản lý tín nhiệm (Credit Manager):

- Duy trì bộ đếm sức chứa của Router kế tiếp. Khối TX chỉ được phép bơm Flit vào mạng khi Credit > 0, triệt tiêu hoàn toàn rủi ro tràn bộ đệm (Drop-free).



2. Giải pháp đề xuất của đề tài

- NETWORK INTERFACE: Slave_{ni}:





2. Giải pháp đề xuất của đề tài

NGUYÊN LÝ HOẠT ĐỘNG SLAVE NI (SHARED RAM NODE)

1. Tích hợp bộ nhớ đồng bộ (Synchronous RAM)

- Quản lý trực tiếp mảng RAM nội bộ (1024x32-bit).
- Tự động dịch mask địa chỉ và hỗ trợ ghi dữ liệu chi tiết đến từng Byte (thông qua tín hiệu Byte Enable).

2. Giải mã và Thực thi lệnh (RX FSM - Chiều nhận)

- **Lệnh Ghi (Write):** Thu thập lần lượt Head Flit (Cờ điều khiển), Body Flit (Địa chỉ) và Tail Flit (Dữ liệu) để chót thẳng vào RAM.
- **Lệnh Đọc (Read):** Nhận Head và Tail (chứa địa chỉ). Khối sẽ "nhớ" tọa độ của Master vừa gửi yêu cầu, sau đó chờ 2 chu kỳ xung nhịp để RAM trích xuất dữ liệu.

3. Đóng gói phản hồi (TX FSM - Chiều gửi)

- Chỉ được kích hoạt khi cần trả kết quả dữ liệu cho lệnh Đọc.
- **Đảo chiều định tuyến:** Lấy tọa độ Master đã lưu làm đích đến, tự động sinh ra 2 Flits (Head mang tọa độ đích $\rightarrow$ Tail mang dữ liệu rdata) để bơm ngược vào NoC.

4. Kiểm soát luồng độc lập (Credit-based)

- Tích hợp bộ đếm Credit riêng rẽ để theo dõi dung lượng Router kế tiếp, đảm bảo việc trả dữ liệu về mạng không bao giờ gây tràn bộ đệm.



2. Giải pháp đề xuất của đề tài

Kiến trúc bus Open Bus interface trong Hệ thống SoC:

1. Đặc điểm cốt lõi (Core Characteristics)

- **Tiêu chuẩn mở (Open-source):** Kiến trúc giao tiếp điểm-điểm (Point-to-point) miễn phí bản quyền, tối ưu hoàn hảo cho hệ sinh thái RISC-V.
- **Tách biệt pha (Phase Separation):** Độc lập hoàn toàn **Pha Yêu cầu** (Request - gửi địa chỉ) và **Pha Phản hồi** (Response - nhận dữ liệu).
- **Đường ống hóa (Pipelined):** Cho phép CPU phát liên tiếp nhiều yêu cầu mà không bị "đóng băng" chờ dữ liệu của lệnh trước trả về.
- **Không dồn kênh (Unmultiplexed):** Đường truyền Address và Data tách biệt, giảm thiểu độ trễ giải mã phần cứng.



2. Giải pháp đề xuất của đề tài

2. Kiến trúc tổng quát (Mô hình Master - Slave)

- **Master (CPU RISC-V):** Đóng vai trò khởi tạo. Cung cấp tín hiệu:
 - req: Báo hiệu có yêu cầu giao dịch.
 - we & addr: Xác định thao tác Đọc/Ghi và vị trí bộ nhớ.
 - wdata: Dữ liệu cần ghi xuống.
- **Slave (Master_NI / Shared RAM):** Đóng vai trò phục vụ. Phản hồi tín hiệu:
 - gnt (Grant): Chấp nhận yêu cầu từ Master (Giải phóng tín hiệu req).
 - rvalid: Cờ báo hiệu dữ liệu đọc đã hợp lệ.
 - rdata: Trả luồng dữ liệu đọc về cho Master.



2. Giải pháp đề xuất của đề tài

3. Kết Nối OBI và RISC-V:

Để cpu có thể hiểu được BUS OBI thì em sử dụng 1 file `cpu_obi_bridge`.

- **1. `cpu_obi_bridge.v` - Trạm Phân Luồng (Cảnh sát giao thông)**
- **Chức năng chính:**
- CPU chỉ biết đọc/ghi dữ liệu chứ không hề biết mạng NoC là gì. Khối Bridge này đứng ngay cửa ngõ CPU, nhìn vào địa chỉ (`cpu_addr`) để quyết định xem dữ liệu sẽ được cất vào RAM nội bộ của Node, hay phải gửi ra mạng NoC.



2. Giải pháp đề xuất của đề tài

- **Cách hoạt động cụ thể:**
- **Chuyển đổi Mask (Byte Enable):** RISC-V xuất ra tín hiệu store_sel (để báo là đang ghi Byte, Half-word hay Word). Bridge sẽ dùng tổ hợp logic case để dịch tín hiệu này sang chuỗi 4-bit be_gen (Byte Enable) cho chuẩn với RAM.
- **Phân dải địa chỉ (Address Decoding):** Dòng wire is_noc = (saved_addr >= 32'h00001000); chính là chốt chặn. Nếu địa chỉ < 0x1000\$, nó bật tín hiệu local_req để cất vào RAM nội bộ. Nếu >= 0x1000\$, nó bật noc_req để đẩy sang Master NI.
- **FSM của Bridge (3 Trạng thái):**
- **IDLE (Chờ việc):** Đứng im chờ tín hiệu cpu_req. Khi có yêu cầu, nó chốt giữ toàn bộ địa chỉ, dữ liệu lại và chuyển sang WAIT_GNT.
- **WAIT_GNT (Chờ cấp quyền):** * Nó giờ tín hiệu Request (noc_req hoặc local_req) lên và chờ đối tác trả lời bằng tín hiệu gnt (Grant).
 - **Nếu là lệnh GHI:** Có gnt nghĩa là đối tác đã lấy xong dữ liệu $\rightarrow$ Xong việc, quay về IDLE và tháo cpu_stall để CPU chạy tiếp lệnh khác.
 - **Nếu là lệnh ĐỌC:** Có gnt mới chỉ là đối tác nhận được yêu cầu, phải chuyển sang WAIT_RVALID để chờ dữ liệu thực tế trả về.
- **WAIT_RVALID (Chờ dữ liệu Đọc):** Đợi tín hiệu rvalid bật lên, nó sẽ ném dữ liệu ngược lại cho CPU và quay về IDLE.

03

KẾT QUẢ CỦA GIẢI PHÁP ĐỀ XUẤT





3. Kết quả của giải pháp đề xuất

Console xuất ra:

```
# =====
# Time: 50000 ns | PC_00: 00000000 | PC_22: 00000000
# Time: 70000 ns | PC_00: 00000004 | PC_22: 00000004
# Time: 90000 ns | PC_00: 00000008 | PC_22: 00000008
# Time: 110000 ns | PC_00: 0000000c | PC_22: 0000000c
# Time: 130000 ns | PC_00: 00000010 | PC_22: 00000010
# Time: 150000 ns | PC_00: 00000014 | PC_22: 00000014
# Time: 170000 ns | PC_00: 00000018 | PC_22: 00000018
# Time: 190000 ns | PC_00: 0000001c | PC_22: 0000001c
# Time: 210000 ns | PC_00: 00000020 | PC_22: 00000020
# Time: 230000 ns | PC_00: 00000024 | PC_22: 00000024
# Time: 250000 ns | PC_00: 00000024 | PC_22: 00000024
# [270000 ns] & [MASTER 0,0 - VIP] Phat dong cuoc dua! Gui payload 14 (0xE) len NoC.
# [270000 ns] & [MASTER 2,2 - Nomal] Phat dong cuoc dua! Gui payload 5 (0x5) len NoC.
# Time: 270000 ns | PC_00: 00000028 | PC_22: 00000028
# [290000 ns] & [MASTER 0,0 - VIP] Phat dong cuoc dua! Gui payload 14 (0xE) len NoC.
# [290000 ns] & [MASTER 2,2 - Nomal] Phat dong cuoc dua! Gui payload 5 (0x5) len NoC.
# Time: 290000 ns | PC_00: 0000002c | PC_22: 0000002c
# [310000 ns] & [MASTER 0,0 - VIP] Phat dong cuoc dua! Gui payload 14 (0xE) len NoC.
# [310000 ns] & [MASTER 2,2 - Nomal] Phat dong cuoc dua! Gui payload 5 (0x5) len NoC.
# Time: 310000 ns | PC_00: 00000030 | PC_22: 00000030
#
# [330000 ns] & [SLAVE RAM 1,1] Nhan duoc Flit thu 1!
# -> Du lieu mang theo (Payload): 0x50fc0000
# Time: 330000 ns | PC_00: 00000034 | PC_22: 00000034
#
# [350000 ns] & [SLAVE RAM 1,1] Nhan duoc Flit thu 2!
# -> Du lieu mang theo (Payload): 0x1100
# Time: 350000 ns | PC_00: 00000038 | PC_22: 00000038
#
# [370000 ns] & [SLAVE RAM 1,1] Nhan duoc Flit thu 3!
# -> Du lieu mang theo (Payload): 0xe
# -> Ket luan: Day la du lieu (14) cua Master (0,0) - Kha nang cao la VIP den truoc!
# Time: 370000 ns | PC_00: 0000003c | PC_22: 0000003c
#
# [390000 ns] & [SLAVE RAM 1,1] Nhan duoc Flit thu 4!
```



3. Kết quả của giải pháp đề xuất

```
# [430000 ns] 8 [SLAVE RAM 1,1] Nhan duoc Flit thu 6!
#         -> Du lieu mang theo (Payload): 0x5
#         -> Ket luan: Day la du lieu (5) cua Master (2,2) - Khoe cham cham thi den sau!
# Time: 430000 ns | PC_00: 00000048 | PC_22: 00000048
# Time: 450000 ns | PC_00: 0000004c | PC_22: 0000004c
# Time: 470000 ns | PC_00: 00000050 | PC_22: 00000050
# Time: 490000 ns | PC_00: 00000054 | PC_22: 00000054
# Time: 510000 ns | PC_00: 00000058 | PC_22: 00000058
# Time: 530000 ns | PC_00: 0000005c | PC_22: 0000005c
# Time: 550000 ns | PC_00: 00000060 | PC_22: 00000060
# Time: 570000 ns | PC_00: 00000064 | PC_22: 00000064
# Time: 590000 ns | PC_00: 00000068 | PC_22: 00000068
# Time: 610000 ns | PC_00: 0000006c | PC_22: 0000006c
# Time: 630000 ns | PC_00: 00000070 | PC_22: 00000070
# Time: 650000 ns | PC_00: 00000074 | PC_22: 00000074
# Time: 670000 ns | PC_00: 00000078 | PC_22: 00000078
# Time: 690000 ns | PC_00: 0000007c | PC_22: 0000007c
# Time: 710000 ns | PC_00: 00000080 | PC_22: 00000080
# Time: 730000 ns | PC_00: 00000084 | PC_22: 00000084
# Time: 750000 ns | PC_00: 00000088 | PC_22: 00000088
# Time: 770000 ns | PC_00: 0000008c | PC_22: 0000008c
#
# =====
# TONG KET BO NHO
# =====
# RAM[65] = 0x00000005
# RAM[64] = 0x0000000e
#
# -> THANH CONG! Chuc nang Uu tien (Priority) hoat dong .
# -> Master 0,0 da duoc Router (1,1) mo cua cho vao truoc. Master 2,2 di sau an toan!
# =====
#
```

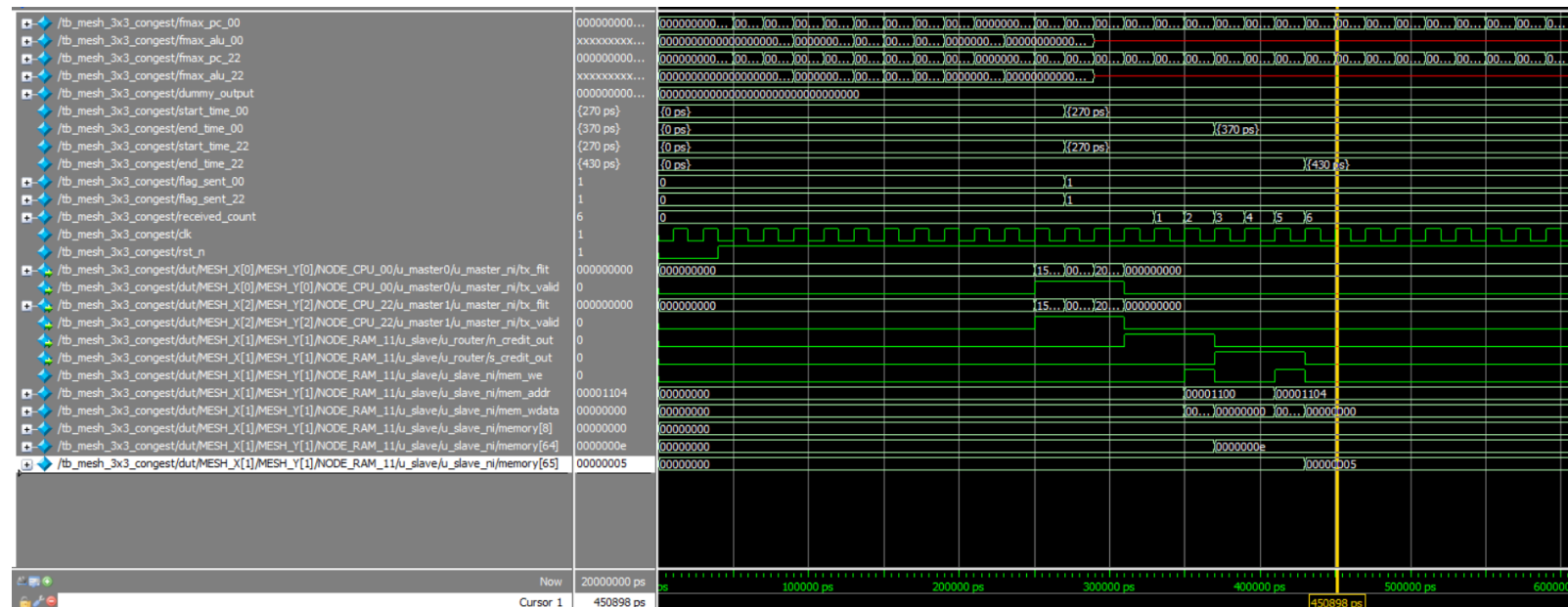


3. Kết quả của giải pháp đề xuất

-Kịch bản 2 test nghẽn mạng:

Th1: Thông thoáng:

L_FACT = 50.





3. Kết quả của giải pháp đề xuất

```
# [CONGESTION & LATENCY TEST] KIEM TRA DO TRE GHE GHEN MANG
# =====
# [270000 ns] & [MASTER 0,0 - VIP] Bat dau roi ben! (Thoi diem xuat phat)
# [270000 ns] & [MASTER 2,2 - Nomal] Bat dau roi ben! (Thoi diem xuat phat)
# [370000 ns] & [SLAVE RAM 1,1] Āa nhan xong du lieu 14 cua VIP (0,0)
# [430000 ns] & [SLAVE RAM 1,1] Āa nhan xong du lieu 5 cua Thuong (2,2)
#
# =====
# TONG KET BO NHO VA DO TRE (LATENCY REPORT)
# =====
# RAM[64] (VIP 0,0 ghi vao) = 0x0000000e
# RAM[65] (Thuong 2,2 ghi vao) = 0x00000005
# -----
# â' DO TRE CUA MASTER (0,0) VIP: 100000 ns (Khoang 5 Chu ky clock)
# â' DO TRE CUA MASTER (2,2) Normal: 160000 ns (Khoang 8 Chu ky clock)
# =====
```



3. Kết quả của giải pháp đề xuất

Tần số Fmax: là 53.08MHZ

Table of Contents

Slow Model Fmax Summary

	Fmax	Restricted Fmax	Clock Name	Note
1	53.08 MHz	53.08 MHz	main_clk	



3. Kết quả của giải pháp đề xuất

Tổng tài nguyên của NoC này là 32,272 logic.

Với 22,850 logic combination và 22,378 logic register.

Table of Contents

Flow Summary

Flow Status	Successful - Tue Jun 23 08:09:04 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	riscV_noc
Top-level Entity Name	mesh_3x3
Family	Cyclone II
Device	EP2C50F672C6
Timing Models	Final
Total logic elements	32,272 / 50,528 (64 %)
Total combinational functions	22,850 / 50,528 (45 %)
Dedicated logic registers	22,378 / 50,528 (44 %)
Total registers	22378
Total pins	162 / 450 (36 %)
Total virtual pins	0
Total memory bits	98,304 / 594,432 (17 %)
Embedded Multiplier 9-bit elements	0 / 172 (0 %)
Total PLLs	0 / 4 (0 %)

04

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN CỦA ĐỀ TÀI





4. Kết luận và hướng phát triển của đề tài

Kết luận

Đề tài đã hoàn thành các mục tiêu đề ra về thiết kế và hiện thực hệ thống Network-on-Chip tích hợp lõi vi xử lý mutiply core.

- Kiến trúc cơ bản của 1 tile.
- Chuẩn hóa bus OBI.
- Hoàn thành kiểm độ ưu tiên, nghẽn mạng.



4. Kết luận và hướng phát triển của đề tài

Hướng phát triển :

Nâng cấp bộ vi xử lý: Sử dụng nhân Ibex (RISC-V) chuẩn công nghiệp

- Tích hợp bộ nhớ đệm cục bộ (L1 Cache)
- Tích hợp Kênh ảo (Virtual Channels - VC) cho Router
- Giao thức đồng bộ bộ nhớ đệm (Cache Coherence - MESI)

công việc tiếp theo:

- Tối ưu hóa NoC để đạt $F_{max} > 100\text{mhz}$ và giảm tổng logic < 25000



Tài liệu tham khảo

- [1] W. J. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA, USA: Morgan Kaufmann, 2004.
- [2] L. Benini and G. De Micheli, "Networks on Chips: A New SoC Paradigm," in *Computer*, vol. 35, no. 1, pp. 70-78, Jan. 2002.
- [3] J. Duato, S. Yalamanchili, and L. Ni, *Interconnection Networks: An Engineering Approach*. Morgan Kaufmann, 2002.
- [4] OpenCores Organization, "Wishbone System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores", Revision B4, 2010. [Online]. Available: https://cdn.opencores.org/downloads/wbspec_b4.pdf

05

HỎI VÀ ĐÁP





Phụ lục (nếu có)

https://github.com/nhuyha2k5/NoC_with_multiply_core